



杭州电子科技大学

实验五 波形发生器实验

课 程 微机原理与接口技术

学 院 自动化学院

专 业 自动化 (新工科试验班)

学 号 24061428

学生姓名 张跃哲

指导教师 金朝阳

完成日期 2026 年 6 月 1 日

目录

1	实验要求与实验目的	1
1.1	实验要求	1
1.2	实验目的	1
1.3	总体设计思路	1
2	Proteus 原理图与硬件电路说明	2
2.1	完整原理图	2
2.2	单片机最小系统	3
2.3	DAC0832 与模拟输出电路	5
2.4	波形选择与 LED 指示电路	6
2.5	频率选择电路	7
2.6	主要元件清单	8
3	软件设计与程序流程	8
3.1	端口分配	8
3.2	主程序流程	9
3.3	四种波形生成方法	9
3.4	正弦波查表流程	10
3.5	延时与频率控制	10
3.6	完整实验代码	10
3.7	关键代码说明	17
4	调试过程与实验结果	17
4.1	调试步骤	17
4.2	四种输出波形	18
4.3	频率调节对比	18
4.4	结果验证表	19
5	问题分析与改进建议	19
6	实验结论	19

1 实验要求与实验目的

1.1 实验要求

本实验要求在 Proteus 中搭建以 AT89C51 和 DAC0832 为核心的波形发生器。单片机通过并行口向 DAC0832 输出 8 位数字量，经运算放大器转换为模拟电压，并在示波器上观察输出波形。基本要求如下：

1. 使用 AT89C51 的 P0 口连接 DAC0832 的 DI0–DI7，完成 8 位数字量到模拟量的转换。
2. 使用 P1.0–P1.3 的拨码开关选择方波、锯齿波、三角波和正弦波四种输出波形。
3. 使用 P1.4–P1.7 的 LED 指示当前选中的波形，LED 采用低电平有效方式驱动。
4. 使用 P2 口作为频率选择输入，通过改变 P2 口拨码值调节软件延时，从而改变输出波形频率。

在完成基本功能的基础上，实验还包含以下拓展要求：

1. 在 Proteus 中完整呈现单片机最小系统、DAC0832 接口、运放输出、波形选择和频率选择电路。
2. 对四种波形分别进行仿真截图，证明输出波形形状与程序设计相符。
3. 对不同 P2 输入下的波形频率进行对比，说明频率控制功能有效。

1.2 实验目的

1. 掌握 AT89C51 并行 I/O 口输出数字量、读取拨码输入以及驱动 LED 指示电路的方法。
2. 理解 DAC0832 的数字量输入、参考电压和运算放大器电流-电压转换输出原理。
3. 掌握用软件查表和循环计数生成周期波形的的方法，并分析软件延时对频率的影响。
4. 提高 Proteus 环境下单片机、D/A 转换器和虚拟示波器联合调试的能力。

1.3 总体设计思路

系统以 AT89C51 为控制核心，P0 口作为波形数据总线，直接把 8 位采样值送入 DAC0832。DAC0832 工作在直通转换方式，片选和写控制信号固定为有效状态，P0 口数据变化后即可在模拟输出端体现为对应电压。P1 口低四位连接波形选择拨码开关，程序每输出一个周期后重新扫描按键；P1 口高四位连接 LED，用于指示当前波形。P2 口连接 8 位拨码开关作为频率控制量，程序将其取反后作为延时循环次数，因此 P2 值越大，延时越短，输出频率越高。

2 Proteus 原理图与硬件电路说明

2.1 完整原理图

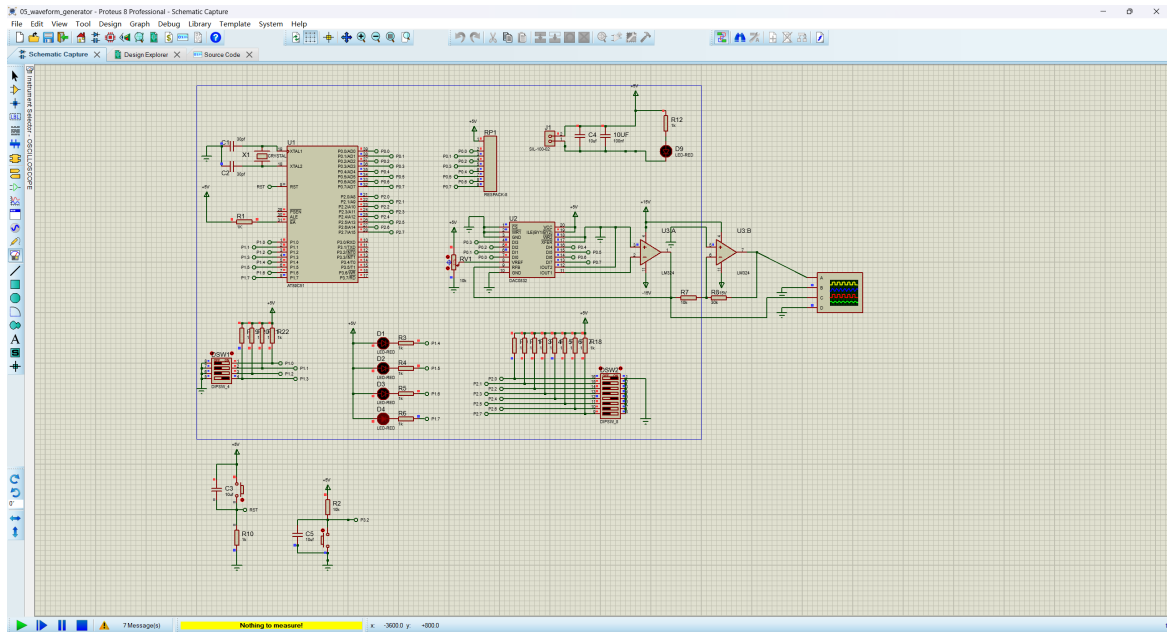


图 1 实验五波形发生器 Proteus 完整原理图

完整电路由 AT89C51 最小系统、DAC0832 数模转换模块、LM324 运算放大器输出模块、P1 口波形选择与 LED 指示模块、P2 口频率选择模块组成。数字部分使用 +5V 供电，运放部分采用正负电源供电，以便输出端能够获得较好的模拟波形摆幅。

2.2 单片机最小系统

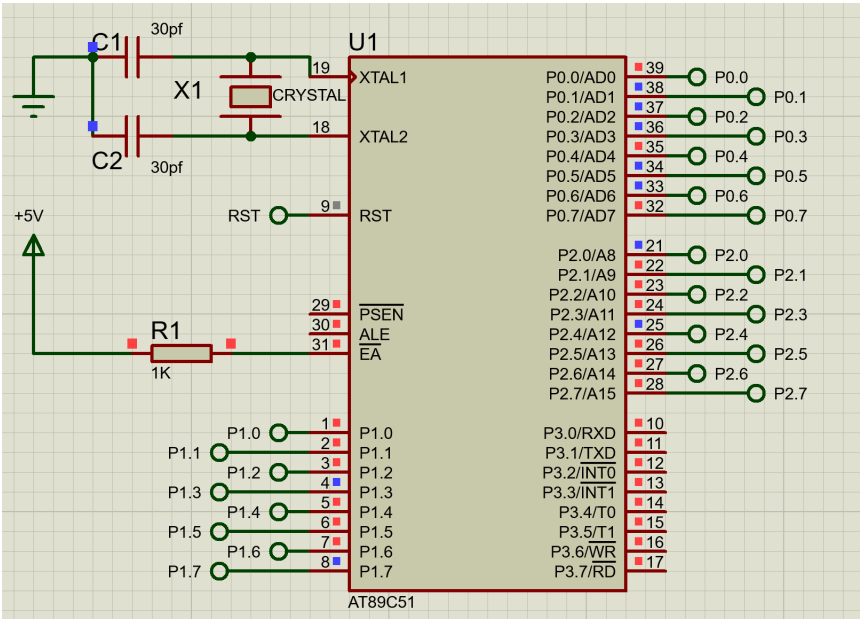


图 2 AT89C51 供电与 12MHz 晶振电路

AT89C51 的 VCC 与 GND 接入 +5V 电源和公共地，XTAL1、XTAL2 外接 12MHz 晶振，并配合两个 30pF 电容构成时钟振荡电路。12MHz 晶振下，8051 一个机器周期约为 1 μ s，便于通过软件循环估算延时时间。

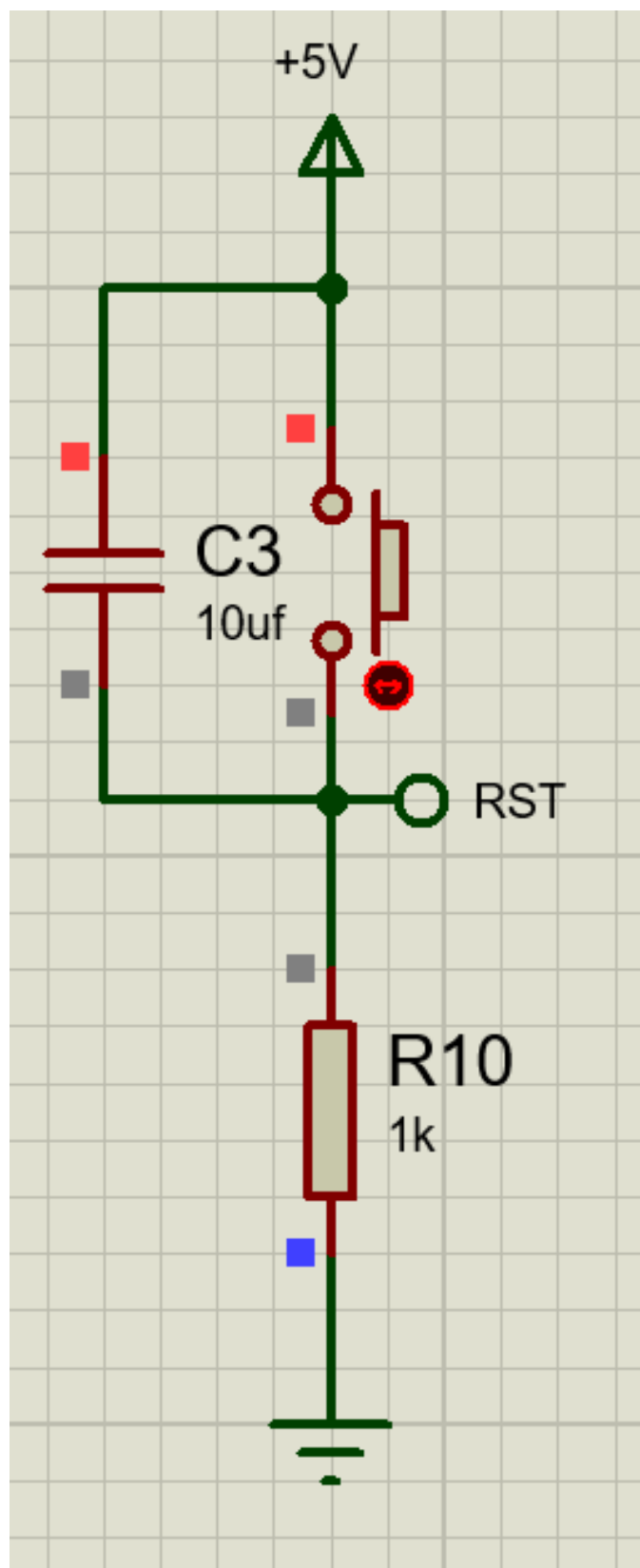


图 3 单片机复位电路

复位电路由按键、电阻和电容构成。上电时电容充电产生短暂高电平，按下复位键时 RST 引脚被拉到有效电平，使程序从 0000H 复位向量重新开始执行。

2.3 DAC0832 与模拟输出电路

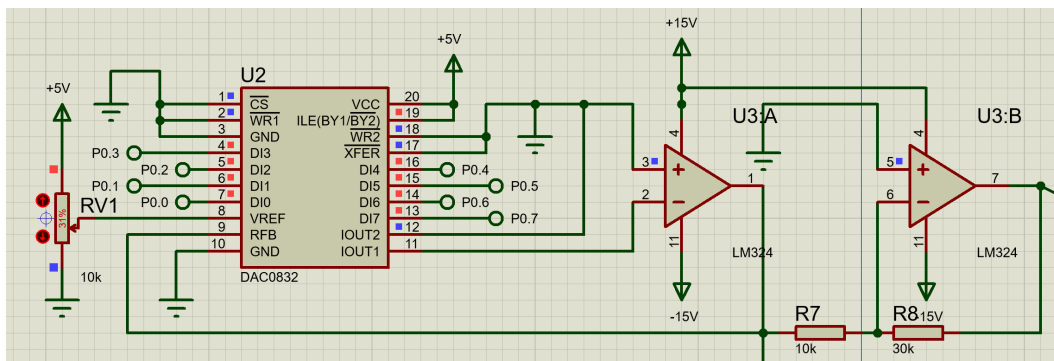


图 4 DAC0832 与 LM324 运放输出电路

DAC0832 的 DI0–DI7 与 AT89C51 的 P0.0–P0.7 相连，作为 8 位并行输入数据总线。电路中 CS、WR1、WR2、XFER 等控制端固定为有效状态，使 DAC0832 处于直通工作方式。单片机写入 P0 的数据不需要额外片选脉冲，即可进入 D/A 转换链路。参考电压由电位器调节，输出电流经 LM324 运算放大器转换为电压信号，最终送到示波器或模拟波形图进行观察。

2.4 波形选择与 LED 指示电路

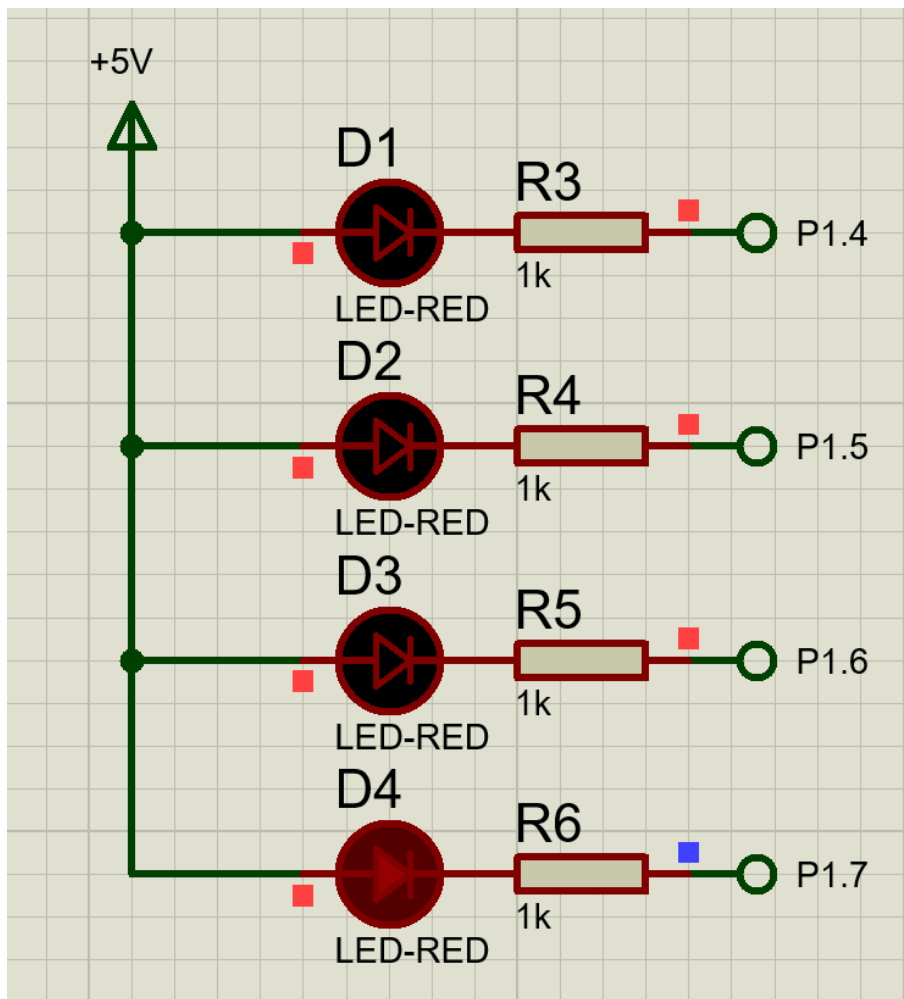


图 5 P1 口波形选择拨码开关与 LED 指示电路

P1.0–P1.3 分别对应方波、锯齿波、三角波和正弦波选择输入，拨码开关闭合时相应引脚为低电平。P1.4–P1.7 分别连接四个 LED，程序输出低电平点亮相应指示灯。由于 P1 口同一端口同时承担输入和输出功能，程序在点亮 LED 时保持低四位为高电平，避免影响下一次按键扫描。

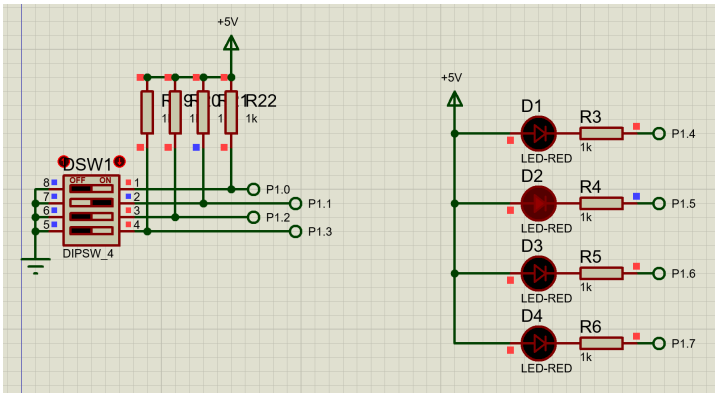


图 6 波形指示 LED 点亮状态

2.5 频率选择电路

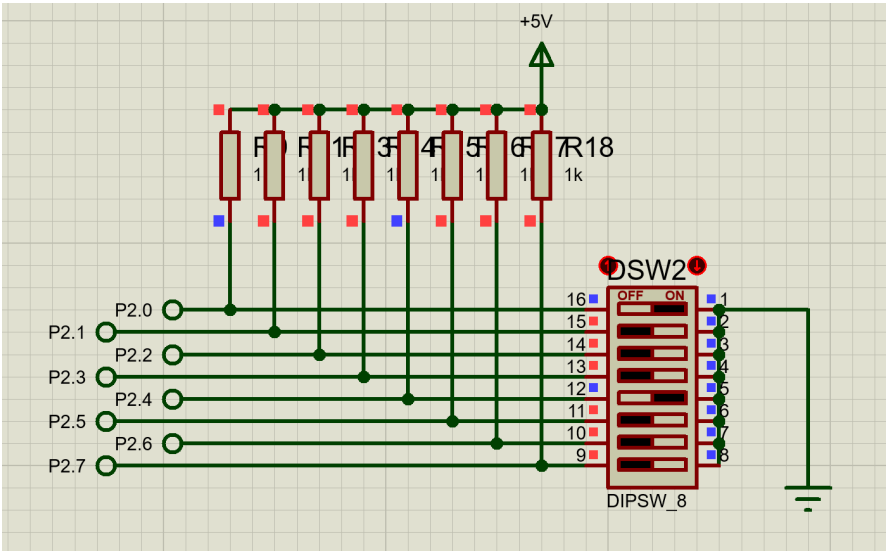


图 7 P2 口 8 位频率选择拨码开关

P2.0-P2.7 连接 DIPSW_8 作为频率输入。程序在延时函数中读取 P2，然后执行取反操作，得到软件延时循环次数。当 P2 值较小时，取反后的延时值较大，波形周期较长；当 P2 值较大时，延时值较小，波形周期缩短。

2.6 主要元件清单

表 1 实验五主要元件与功能

序号	元件名称	型号或规格	功能说明
1	单片机	AT89C51	读取拨码开关，输出波形采样值，控制 LED 指示
2	数模转换器	DAC0832	将 P0 口 8 位数字量转换为模拟电流输出
3	运算放大器	LM324	完成电流-电压转换及模拟电压输出
4	晶振	12MHz	提供单片机运行时钟
5	电位器	POT-HG	调节 DAC0832 参考电压，影响输出幅度
6	拨码开关	DIPSW	P1 选择波形，P2 调节频率
7	LED 与电阻	LED / RES	指示当前输出波形并限制 LED 电流
8	示波器或波形图	OSCILLOSCOPE / GRAPH	观察运放输出端模拟波形

3 软件设计与程序流程

3.1 端口分配

表 2 AT89C51 端口分配表

端口	连接对象	软件作用
P0.0–P0.7	DAC0832 DI0–DI7	输出 8 位波形采样值
P1.0	方波选择开关	低电平有效，返回波形编号 00H
P1.1	锯齿波选择开关	低电平有效，返回波形编号 01H
P1.2	三角波选择开关	低电平有效，返回波形编号 02H
P1.3	正弦波选择开关	低电平有效，返回波形编号 03H
P1.4–P1.7	四个 LED 指示灯	低电平点亮，显示当前波形
P2.0–P2.7	频率选择拨码开关	输入延时调节量，P2 越大频率越高

3.2 主程序流程

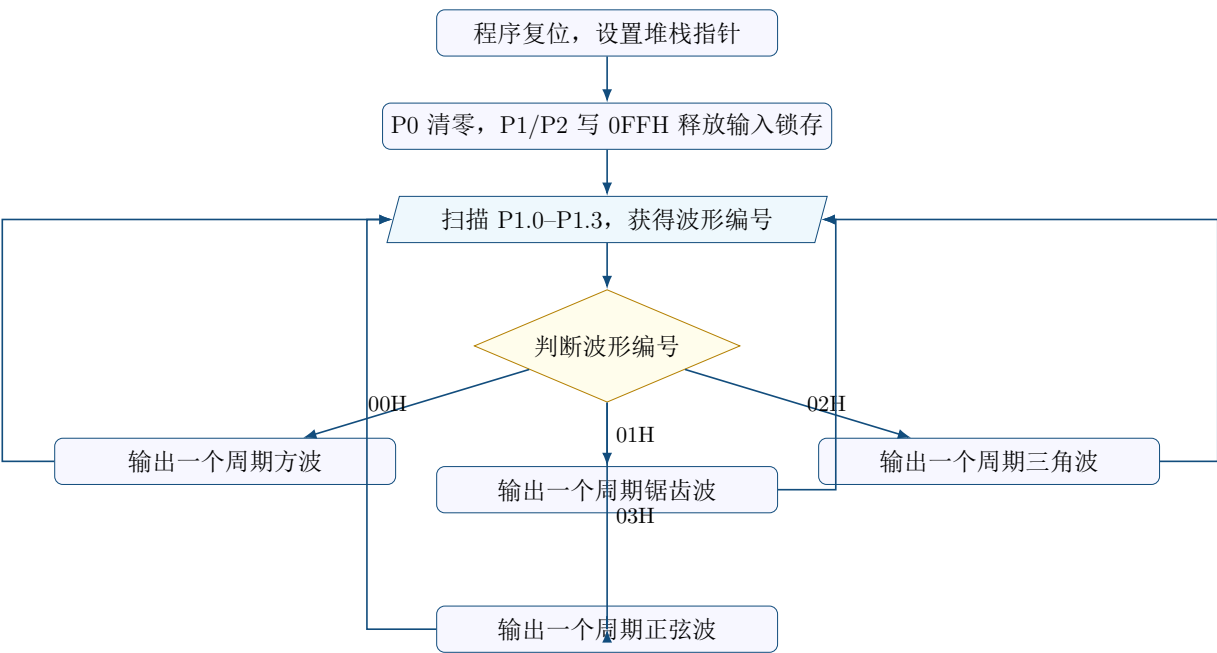


图 8 波形发生器主程序流程图

主程序采用循环扫描方式。每次调用对应波形子程序时只输出一个完整周期，周期结束后返回主循环重新读取 P1 低四位。这样设计的好处是按键切换不需要中断支持，程序结构清晰，且能够在一个周期结束后稳定切换到新的波形。

3.3 四种波形生成方法

表 3 四种波形的软件生成方法

波形	输出规律	程序实现
方波	00H 与 FFH 交替输出	低电平保持半周期，高电平保持半周期，形成高低电平交替
锯齿波	00H 到 FFH 单调递增	累加器从 00H 自增到溢出，每步输出一次并延时
三角波	00H 到 FFH 再回到 00H	先执行递增输出，再执行递减输出，形成线性上升和下降
正弦波	以 80H 为中心上下变化	读取四分之一正弦表，通过对称和取反生成完整周期

3.4 正弦波查表流程

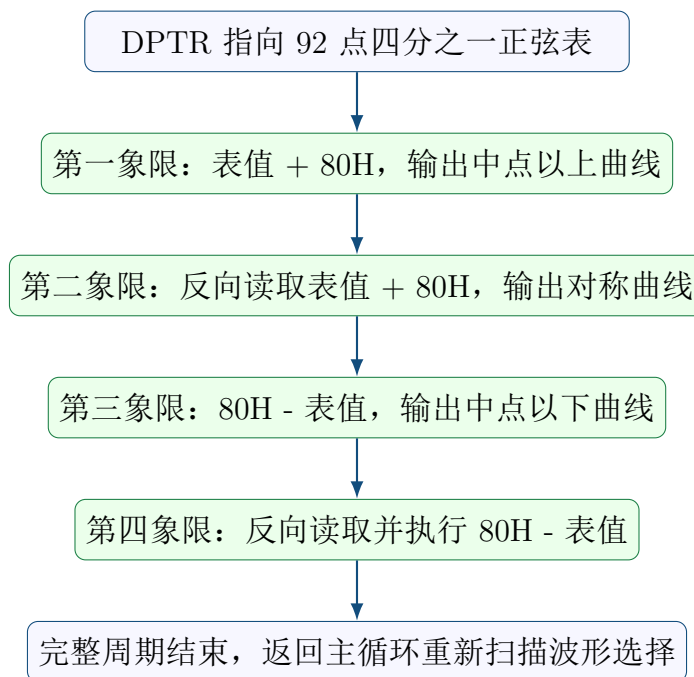


图 9 正弦波查表与对称生成流程图

正弦表中仅保存从 0 到峰值的四分之一周期幅值，范围为 00H–7FH。程序输出时以 80H 作为中心值，前两个象限采用加法生成上半周，后两个象限采用减法生成下半周。与直接存储完整周期相比，这种方法节省了代码空间，同时保证输出曲线平滑。

3.5 延时与频率控制

延时函数 STEP_DELAY 是频率控制的核心。程序读取 P2 后先取反，得到 255 - P2，再作为 R6 的循环次数。若 P2 为 FFH，取反结果为 00H，程序通过 INC A 保证仍有最小延时。其关系可概括为：

$$T_{\text{step}} \propto 255 - P2$$

因此 P2 值越大，每个采样点之间的间隔越短；波形每周期采样点数不变时，周期缩短，频率升高。方波使用 HALF_DELAY 形成半周期延时，锯齿波、三角波和正弦波则在每个采样点后调用 STEP_DELAY。

3.6 完整实验代码

Listing 1 05_waveform_generator.A51 完整源代码

```

1 ;=====
2 ; Experiment 5: AT89C51 + DAC0832 waveform generator
3 ; Compiler: ASEM-51 (Proteus)

```

```

4 ;
5 ; P0.0-P0.7 -> DAC0832 DIO-DI7
6 ; P1.0      -> square wave select, low active
7 ; P1.1      -> sawtooth wave select, low active
8 ; P1.2      -> triangle wave select, low active
9 ; P1.3      -> sine wave select, low active
10 ; P1.4     -> square wave LED, low active
11 ; P1.5     -> sawtooth wave LED, low active
12 ; P1.6     -> triangle wave LED, low active
13 ; P1.7     -> sine wave LED, low active
14 ; P2       -> frequency input, larger value means higher frequency
15 ;=====
16
17 $NOMOD51
18 $INCLUDE (8051.MCU)
19
20 ;=====
21 ; Reset vector
22 ;=====
23
24     ORG 0000H
25     LJMP START
26
27 ;=====
28 ; Main program
29 ;=====
30
31     ORG 0100H
32
33 START:
34     MOV SP,#60H
35     MOV P0,#00H
36     MOV P2,#0FFH      ; release P2 input latch
37     MOV P1,#0FFH      ; release P1.0-P1.3 and turn LEDs off
38
39 MAIN:
40     ACALL SCAN_KEY
41
42     CJNE A,#00H,CHK_SAW
43     ACALL SQUARE_WAVE

```

```

44         SJMP MAIN
45
46 CHK_SAW:
47         CJNE A,#01H,CHK_TRI
48         ACALL SAW_WAVE
49         SJMP MAIN
50
51 CHK_TRI:
52         CJNE A,#02H,CHK_SIN
53         ACALL TRI_WAVE
54         SJMP MAIN
55
56 CHK_SIN:
57         ACALL SINE_WAVE
58         SJMP MAIN
59
60 ;=====
61 ; Scan waveform select key
62 ; Return A:
63 ; 00H square, 01H sawtooth, 02H triangle, 03H sine
64 ; If no key is pressed, square wave is selected by default.
65 ;=====
66
67 SCAN_KEY:
68         MOV A,P1
69         ANL A,#0FH
70         JNB ACC.0,KEY_SQUARE
71         JNB ACC.1,KEY_SAW
72         JNB ACC.2,KEY_TRI
73         JNB ACC.3,KEY_SIN
74
75 KEY_SQUARE:
76         MOV A,#00H
77         RET
78
79 KEY_SAW:
80         MOV A,#01H
81         RET
82
83 KEY_TRI:

```

```

84         MOV A,#02H
85         RET
86
87 KEY_SIN:
88         MOV A,#03H
89         RET
90
91 ;=====
92 ; Waveform output routines
93 ; Each routine outputs one cycle, then returns to MAIN to rescan keys.
94 ;=====
95
96 SQUARE_WAVE:
97         MOV P1,#0EFH      ; P1.4 LED on, P1.0-P1.3 stay high
98         MOV P0,#00H
99         ACALL HALF_DELAY
100        MOV P0,#0FFH
101        ACALL HALF_DELAY
102        RET
103
104 SAW_WAVE:
105        MOV P1,#0DFH      ; P1.5 LED on
106        MOV A,#00H
107 SAW_LOOP:
108        MOV P0,A
109        ACALL STEP_DELAY
110        INC A
111        JNZ SAW_LOOP
112        RET
113
114 TRI_WAVE:
115        MOV P1,#0BFH      ; P1.6 LED on
116        MOV A,#00H
117 TRI_UP:
118        MOV P0,A
119        ACALL STEP_DELAY
120        INC A
121        JNZ TRI_UP
122
123        MOV A,#0FFH

```

```

124 TRI_DOWN:
125     MOV PO,A
126     ACALL STEP_DELAY
127     DEC A
128     JNZ TRI_DOWN
129     MOV PO,#00H
130     ACALL STEP_DELAY
131     RET
132
133 SINE_WAVE:
134     MOV P1,#07FH           ; P1.7 LED on
135
136     MOV DPTR,#SINE_TAB
137     MOV RO,#00H
138 SIN_UP:
139     MOV A,RO
140     MOVC A,@A+DPTR
141     ADD A,#80H
142     MOV PO,A
143     ACALL STEP_DELAY
144     INC RO
145     CJNE RO,#5CH,SIN_UP ; 92 table values
146
147 SIN_BACK_MID:
148     DEC RO
149     MOV A,RO
150     MOVC A,@A+DPTR
151     ADD A,#80H
152     MOV PO,A
153     ACALL STEP_DELAY
154     CJNE RO,#00H,SIN_BACK_MID
155
156     MOV RO,#00H
157 SIN_DOWN:
158     MOV A,RO
159     MOVC A,@A+DPTR
160     MOV B,A
161     MOV A,#80H
162     CLR C
163     SUBB A,B

```

```

164         MOV P0,A
165         ACALL STEP_DELAY
166         INC R0
167         CJNE R0,#5CH,SIN_DOWN
168
169 SIN_BACK_ZERO:
170         DEC R0
171         MOV A,R0
172         MOVC A,@A+DPTR
173         MOV B,A
174         MOV A,#80H
175         CLR C
176         SUBB A,B
177         MOV P0,A
178         ACALL STEP_DELAY
179         CJNE R0,#00H,SIN_BACK_ZERO
180         RET
181
182 ;=====
183 ; Delay routines
184 ; P2 = 255 gives the shortest delay, P2 = 0 gives the longest delay.
185 ;=====
186
187 STEP_DELAY:
188         PUSH ACC
189         MOV A,P2
190         CPL A                ; A = 255 - P2
191         JNZ STEP_SET
192         INC A                ; minimum delay when P2 = 255
193 STEP_SET:
194         MOV R6,A
195 STEP_D1:
196         NOP
197         DJNZ R6,STEP_D1
198         POP ACC
199         RET
200
201 HALF_DELAY:
202         MOV R5,#80
203 HALF_D1:

```

```
204      ACALL STEP_DELAY
205      DJNZ R5, HALF_D1
206      RET
207
208      ;=====
209      ; 92-point quarter sine amplitude table, 0 -> 127.
210      ; One full sine cycle is generated around code 80H.
211      ;=====
212
213 SINE_TAB:
214      DB 00H,02H,04H,07H,09H,0BH,0DH,0FH
215      DB 11H,14H,16H,18H,1AH,1CH,1EH,21H
216      DB 23H,25H,27H,29H,2BH,2DH,2FH,31H
217      DB 33H,35H,37H,39H,3BH,3DH,3FH,41H
218      DB 43H,44H,46H,48H,4AH,4CH,4DH,4FH
219      DB 51H,53H,54H,56H,57H,59H,5BH,5CH
220      DB 5EH,5FH,61H,62H,63H,65H,66H,67H
221      DB 69H,6AH,6BH,6CH,6DH,6EH,6FH,70H
222      DB 71H,72H,73H,74H,75H,76H,77H,78H
223      DB 78H,79H,7AH,7AH,7BH,7BH,7CH,7CH
224      DB 7DH,7DH,7DH,7EH,7EH,7EH,7FH,7FH
225      DB 7FH,7FH,7FH,7FH
226
227      ;=====
228      END
```

3.7 关键代码说明

表 4 关键程序段与功能说明

指令或程序段	功能说明
MOV P0,#00H	初始化 DAC 输入数据，使模拟输出从低电平开始。
MOV P2,#0FFH	向 P2 写 1，释放准双向口输入锁存，便于读取拨码开关。
MOV P1,#0FFH	释放 P1 低四位输入，同时关闭高四位 LED。
ANL A,#0FH	只保留 P1.0–P1.3 的波形选择输入。
JNB ACC.0,KEY_SQUARE	检测低电平有效的方波选择开关。
MOV P1,#0EFH	P1.4 输出低电平，点亮方波指示 LED。
INC A / JNZ SAW_LOOP	锯齿波从 00H 递增到 FFH，溢出后结束一个周期。
MOVC A,@A+DPTR	按偏移量读取程序存储器中的正弦表数据。
CPL A	将 P2 输入取反，把较大的 P2 值转换为较短的软件延时。

4 调试过程与实验结果

4.1 调试步骤

1. 检查 AT89C51 的 VCC、GND、晶振和复位连接，确认单片机能够正常启动。
2. 检查 P0 到 DAC0832 DI0–DI7 的连接顺序，避免位序接反导致波形幅值异常。
3. 确认 DAC0832 控制端处于直通有效状态，LM324 供电和反馈连接正确。
4. 将汇编程序加载到 AT89C51，运行仿真后先观察默认方波输出。
5. 依次切换 P1.0–P1.3 拨码开关，观察四个 LED 指示状态和示波器波形。
6. 保持同一种波形，改变 P2 口拨码值，比较低频、中频和高频时波形周期变化。

4.2 四种输出波形

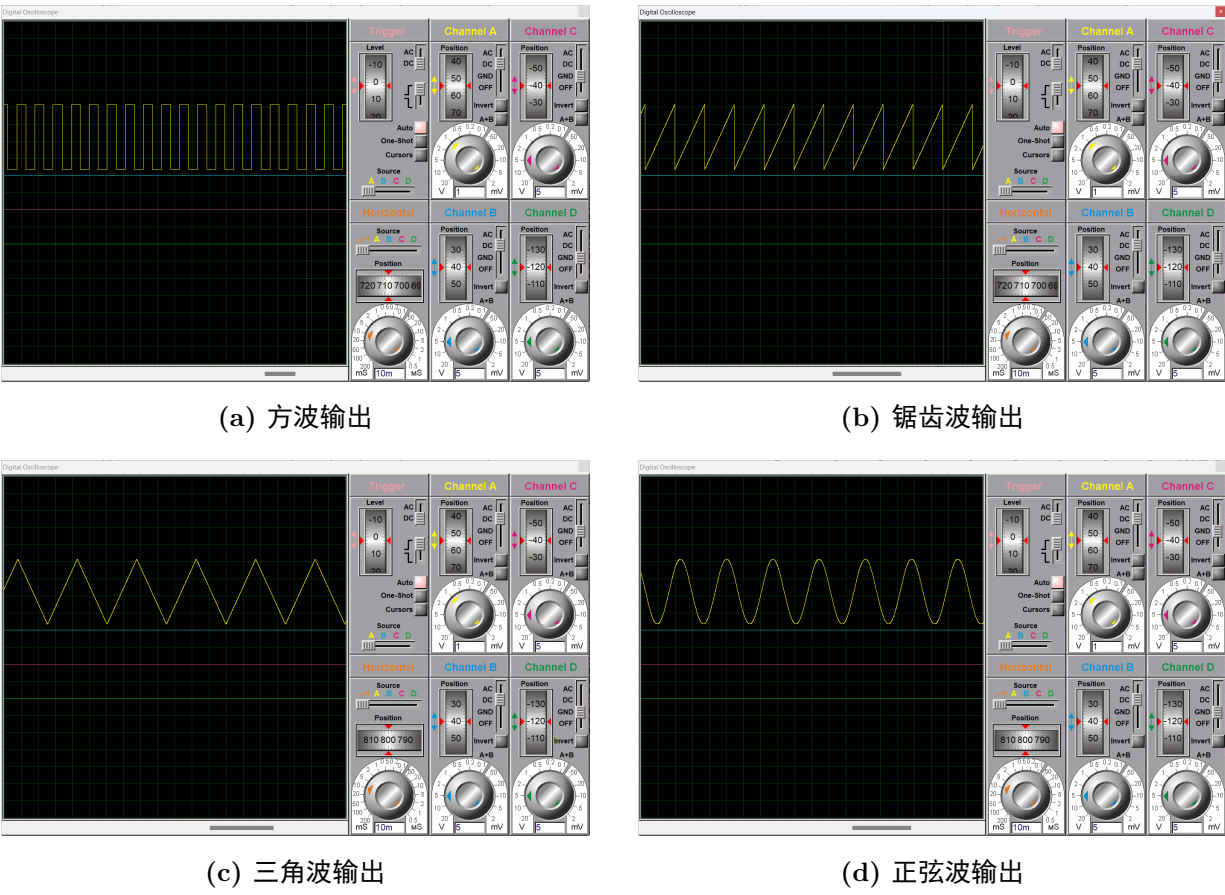


图 10 四种波形的示波器仿真结果

从示波器截图可以看出，方波在高低电平之间快速切换，锯齿波呈现线性上升后跳变回零，三角波具有近似对称的上升和下降边沿，正弦波则表现为连续平滑的周期曲线。四种波形均与程序输出规律一致。

4.3 频率调节对比

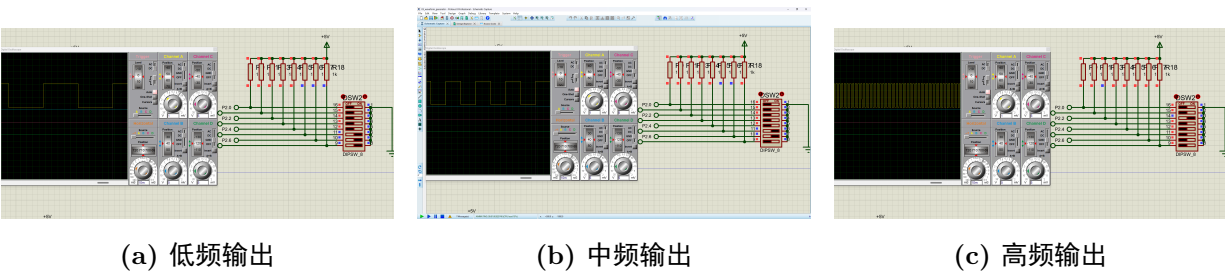


图 11 不同 P2 取值下的频率调节效果

频率对比结果表明，调节 P2 口拨码值后，同一屏幕时间范围内显示的周期数发生明显变化。低频时波形周期长、分布较稀疏；高频时周期变短、波形更密集，说明 P2 口频率控制逻辑有效。

4.4 结果验证表

表 5 波形发生器仿真结果记录

序号	测试内容	预期现象	结果
1	上电复位与默认状态	未按键时默认输出方波	符合
2	方波选择	输出高低电平交替的方波，P1.4 LED 点亮	符合
3	锯齿波选择	输出线性上升并回零的锯齿波，P1.5 LED 点亮	符合
4	三角波选择	输出线性上升再下降的三角波，P1.6 LED 点亮	符合
5	正弦波选择	输出平滑正弦曲线，P1.7 LED 点亮	符合
6	P2 频率调节	P2 值增大时输出频率升高	符合

5 问题分析与改进建议

1. 当前频率调节依赖软件延时，延时期间 CPU 不能处理其他任务，且实际频率受指令执行时间影响。若需要更稳定的频率，可使用定时器中断产生固定采样节拍。
2. DAC0832 采用直通方式，电路简单、便于实验观察，但如果后续需要与多路外设共享数据总线，可以改用锁存控制方式，提高总线时序的可控性。
3. P1 口同时承担开关输入和 LED 输出，程序需要注意写端口时保持低四位为 1。若硬件资源允许，可以将输入和指示灯分配到不同端口，降低端口复用带来的软件约束。
4. 正弦波由 92 点四分之一表生成，曲线已经较平滑。若追求更高精度，可增加表点数量；若追求更高频率，可减少表点或使用定时器配合查表输出。
5. 机械拨码开关在实际硬件中可能存在抖动。本实验在 Proteus 中影响较小，若移植到实物系统，可增加简单消抖或在周期结束时多次确认输入状态。

6 实验结论

本实验完成了基于 AT89C51、DAC0832 和 LM324 的波形发生器设计与仿真。系统通过 P0 口输出数字采样值，经 DAC0832 和运算放大器转换为模拟电压；通过 P1 口选择方

波、锯齿波、三角波和正弦波，并用 LED 指示当前波形；通过 P2 口改变软件延时，实现输出频率调节。仿真结果显示，四种波形形状清晰，频率变化效果明显，硬件连接和软件控制逻辑均满足实验要求。